

RealityFlow: Open-Source Multi-User Immersive Authoring

John T. Murray*

University of Central Florida
Orlando, Florida, USA

ABSTRACT

Immersive Authoring remains primarily driven by user-generated content on social VR applications. This paper introduces and motivates an open-source tool, RealityFlow, implemented on top of the Unity engine, and which supports studying immersive authoring paradigms through visual programming languages. This paper describes the context of visual programming environments as well as the requirements, technical implementation, and overall motivations and goals for future iterations.

Keywords: Virtual reality, authoring tools, visual programming languages.

Index Terms: Human-centered computing—Collaborative and social computing—Collaborative and social computing systems and tools—Open source software; Human-centered computing—Collaborative and social computing—Collaborative and social computing systems and tools—Synchronous editors

1 INTRODUCTION

Platforms depend on the creativity and contributions of a broad community that extend beyond the technologists that build them. The Flash Platform, for instance, provided a foundational environment for creatives, including artists and writers, to experiment and grow while producing works that could be run directly on the web and which didn't require tweaking based on browser implementations [1]. The capabilities of Flash and its language, ActionScript, eventually became the requirements for what we see now in ES2016 and its authoring tools were the main precursor to contemporary game engine development environments, such as Unity.

Some of the most significant factors that have been identified are a lack of role models, a lack of connection of computer science curriculum to interests in doing good for society, and the self-efficacy that may have resulted from negative stereotypes about who is suited to work in computational fields or what the culture is like [2]. Another key factor in is the challenge of teaching computational thinking [3], or computational literacies, especially as learning programming is difficult for novices [4], [5]. Over the past two decades, the increasing affordance and power of computing hardware has been used to address these issues, including through novel programming languages and accessible robotics platforms. Recently, immersive headsets have become widely available and affordable outside of research labs, opening up increased interest in VR applications across several industries and fields. Immersive media can act as a bridge, transforming the negative stereotypes of previous depictions of computing into ones that center social interaction and positive impacts, such as the interfaces shown in *Black Panther*. The radically different modality of immersive media platforms break

from previous computing form factors, such as smartphones [6], and provide a novel sense of presence and engagement [7]. However, these emerging platforms require advanced game engine and programming skills to develop for. Access is, therefore, unequally distributed.

For virtual reality, there is an urgent need to create inviting and powerful tools for creators that do not require them to acquire professional game development tools or the requisite skills [8]. Because VR and Mixed Reality (immersive AR, often associated with the Microsoft HoloLens) are both valuable and not a good fit for traditional development tools and workflows, the idea of *immersive authoring* has grown in popularity [9]–[11]. The powerful vision of creating and experiencing creation directly is probably best depicted in the Star Trek Holodeck.

This paper introduces RealityFlow, an authoring environment built on top of the Unity game engine, and which specifically supports collaborative immersive authoring at run time using a variety of paradigms. The paper is structured as follows: Visual programming languages are described and introduced, followed by a review of current immersive authoring tools and their respective advantages. Next, we present the architecture and features alongside the project goals. Finally, future work in the area and planned features for the next iteration are described.

2 VISUAL PROGRAMMING LANGUAGES

Visual languages are 2-dimensional analogues to traditional languages. There are many visual languages outside of computing, with the visual notation developed over the centuries for music being a prominent example. Visual languages are particularly important to immersive authoring, as they better match the input affordances over text-based interfaces built around the affordances of mouse and keyboard text-based interfaces.

The traditional definition of a *visual language* is one where information is encoded in more than one dimension [12], and which can be divided into visual programming languages (which are executable) and the more broad term visual languages (including Universal Modeling Language (UML) diagrams among others). These are in contrast to *textual languages*, which can be parsed as a stream of characters or tokens into an abstract syntax tree representing the structure of the program. This has led to a number of efforts to build visual programming languages in order to reduce the barrier to entry, with a focus on the benefits to “end users” [13] and “student engagement [14]”. Indeed, several visual programming languages have become standard methods of introducing computer programming. Visual programming languages address the barrier represented by syntax, but they do not themselves include the entire solution. Not only do beginners need to input the program, they must also consider the problem and assess the solution [15].

For the purposes of learning, VPLs offer one solution to overcoming the need for memorization or the constant syntax errors. Scratch [16], for instance, and Blockly [17] are used in schools worldwide and enjoy thriving communities of developers and users. Some VPLs have proven successful outside of education: The Unreal Engine development environment includes a VPL called Blueprints, while Unity recently acquired visual scripting product Bolt (and rebranded it as Unity Visual Scripting). Both offer alternative views on manipulating game

* email: jtm@ucf.edu

assets and engine APIs than the traditional programming languages, C# or C++. In IoT, IBM built NodeRED[18] as a way to wire together devices using a visual programming interface. These languages have proven promising with regard to broadening participation in computing [19].

2.1 Commercial Authoring Tools and Products

Recently, several companies have created tools that offer a different approach to the challenge. These XR content authoring tools can be divided into two categories: professional-oriented and amateur-oriented. Professional-oriented tools include Unity MARS, Unreal VR Mode, Adobe Aero, and Zappar. Amateur-oriented tools target creators who are not professional developers or designers and include Zoe, Facebook Horizons, Neos VR [23], and Rec Room.

Academic researchers have developed several systems that have not been released that incorporate immersive authoring. Zheng and Oney not only help to demonstrate both the utility and history of the term “immersive authoring,” but provide one of the only available software implementations in the form of a VR visual programming application implemented in A-Frame, FlowMatic [10]. FlowMatic is closest in spirit and implementation to the proposed system, with many features proposed that are not on the roadmap for RealityFlow. RealityFlow’s goals were to facilitate collaboration and to support multiplatform editing of XR content, which has most recently been achieved through Rec Room’s authoring tools. However, none of these tools have gained popularity or widespread use as projects such as or Processing [24] or Scratch [16].

Immersive media development necessarily builds on previous screen-based authoring paradigms. First, the authoring metaphors are largely synonymous with the technical affordances of models and scripting, even employed in an immerse mode. MacIntyre and Smith’s description of the efforts to standardize development and implementation of XR technologies is only one side of the challenge of discovering and testing alternative authoring approaches for new media [25].

2.1.1 NeosVR

NeosVR, and its LogiX visual scripting dataflow language, is embedded in a social virtual world application, allowing multiple users to co-create content together in real-time in the same context that the content would be published. Founded in 2014, NeosVR is open with its ambitions to become a metaverse platform in the tradition of Second Life [23]. The *in situ* authoring tool enables users to author behaviors and place content in their own scenes, using the same object-component model that Unity uses but with a custom implementation for behaviors. One of the main drawbacks of NeosVR is shared with the other commercial options here, which is the limitations on how the platform can be used outside of consumption and sharing within the virtual world itself. There are no export options and no way to change underlying design decisions, which limits opportunities for research studies.

2.1.2 Facebook Horizons

Facebook has released a beta of a content creation tool and social sharing platform. The scripting environment uses a blocks-based programming approach situated on floating planes coupled with direct selection and manipulation of the virtual geometry. It incorporates basic events and concepts that are intrinsic to the multi-user VR domain, much as the Altspace VR world building program [26] and its library, the Microsoft Mixed Reality Extension provide. It is even possible to bring in Unity content directly. The primary concern is the use of Facebook accounts in educational settings and the proprietary nature of the ecosystem. Like NeosVR, there is no access to the source code and no way to

bring Horizons-created content out of the Facebook app. The scripting engine also has limits in terms of the number of blocks of any given script.

2.1.3 Zoe

Zoe is a 3D immersive authoring program available on the Oculus Quest and PCVR platforms. It was built for an education audience on top of Unity and further includes plugins for use within the Unity development engine. It is developed by Apelab, a startup founded by Emilie Joly, Maria Beltran and Sylvain Joly in 2014 (Benzakein, 2016). Like most other tools presented here, Zoe supports placing, scaling and moving 3D assets. Zoe departs from other static authoring tools, such as Microsoft Maquette [27] and Google Blocks [28], and Tiltbrush, through its support for behaviors using triggers and actions. Conceptually similar to IFTTT, Zoe supports assigning responses to conditions that trigger on events directly to objects in the scene via a property view. This limits the overall complexity of programs, as only a single relationship can be edited at one time but is considerably simpler.

2.1.4 Rec Room

Like Roblox and AltspaceVR, Rec Room is a multi-platform social virtual world where users can create and experience content other users have created across instanced rooms. The application introduced their visual programming language through the maker pen in 2017¹, extending to the visual programming language they call “circuits” and most recently, V2. Like the work by Kao et al. in Hack.VR [11], nodes are represented as objects in space with ports and wires connecting them. These nodes function as adders or event detectors and drive different effects in the environment.

3 REALITYFLOW

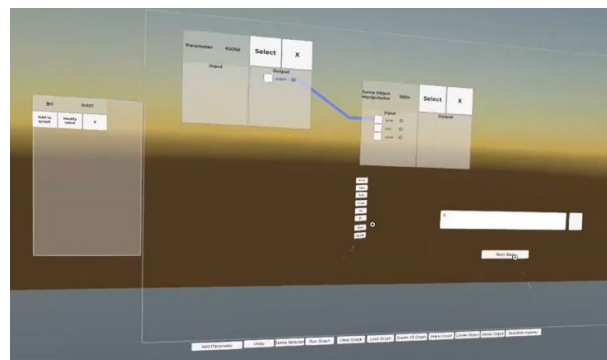


Figure 1: RealityFlow Prototype Whiteboard Interface. Left: Parameters (input). Center: Nodes with input and output ports. Bottom: Graph system commands

RealityFlow addresses the gap in availability of open-source projects by implementing three core features: a dataflow-based VPL immersive editor, a multi-user real-time server for synchronous editing, and cross-platform support across mobile devices and immersive headsets. The platform’s goal is to enable researchers and educators to compare different visual and interaction choices independently from the underlying content representation. This means addressing the authoring lifecycle, including tutorials, different workflows, and exporting and sharing them via a server and database. Additionally, in order to be used in a variety of research scenarios, the choice of building on Unity allows for integration with a variety of plugins and tools

¹ <https://steamcommunity.com/games/RecRoom/announcements/detail/1500120099374725266>

while allowing for server-side libraries and telemetry to be added on top of the content storage.

The project was motivated in part by the lack of educational tools to introduce students to both programming and VR, and the difficulty in comparing the available tool design choices, both commercial and open source, in a controlled experiment.

One of the main decisions that distinguishes this project from others is the choice to build on top of the Unity game engine. While building a solution out of entirely open source software would be ideal, the availability of a variety of assets and plugins make either Unity or Unreal viable solutions.

3.1 RealityFlow Architecture

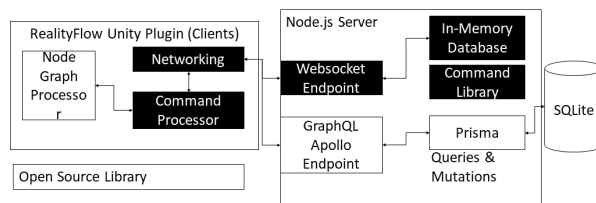


Figure 2: System Architecture

The system consists of three main components, shown in Figure 2: a Unity plugin which contains different visual interfaces for each supported platform (Unity, Mobile and VR currently). The Node.js server, which handles real-time updates and commands through a WebSocket API and a GraphQL endpoint. Finally, the data is persisted in a database.

Collaboration and flexible interaction design were the two core pillars of the project. Ephemeral changes, such as a dragged object transform update, are broadcast to all connected clients and not persisted. The client plugin has different mode-specific UI (e.g. touch controls for mobile, 6DOF controls for VR interactions with script elements, and panels for the Unity IDE plugin). The plugin builds on the open-source library NodeGraphProcessor² to handle execution of nodes, creating visual representations of data, ports and edges for each. The library already offers an editor plugin, but RealityFlow implements a VR and mobile interface for editing these graphs and sharing them between multiple users. The plugin communicates to the server via JSON-formatted WebSocket messages for real-time updates, while significant changes are handled via a GraphQL mutation. Interactions, including selections and movement of nodes, are updated live, including manipulations of the whiteboard depicted in Figure 1.

3.2 Planned Features

The currently available version of RealityFlow³ demonstrates the possibilities, but it currently lacks sophistication in the interface and in the types of content that can be placed or created. The next set of features planned include direct creation of meshes and assignment of typical XR properties through Unity's XR Interaction Toolkit. This will allow authors to directly handle grab events and describe what parts of a scene are potential teleport targets. Additionally, a scratch-based interface is planned to allow direct comparison between the two paradigms, along with a more modular library for blocks and nodes. It is still an outstanding question as to what features support authoring tasks in virtual environments and making these elements easily accessible to

researchers is a major goal alongside collecting data and supporting research studies. The current project uses a proprietary audio solution, Dissonance, which will be replaced with an appropriate open-source alternative, but for now those features will be made optional.

Based on surveying the existing platforms available, the following features are planned for this next iteration:

Automated layout: Previous studies have indicated that manually moving nodes and making connections may detract from the usability. Based on the results, we would implement a standard layout algorithm based on the location of a connection and associated nodes. This concern of organizing connections is critical as it is a chief criticism of data-flow based visualizations. One option would be to offer multiple views on organization and connections, and not constrain ports to a single spatial location relative to objects they correspond to.

Contextual Filtering and Auto-Complete: It's clear that visualizing all parts of a diagram contributes to the cluttered perception of the interface, obscuring valuable details. When navigating a text editor, contextual information such as corresponding symbols are highlighted. In a similar way, VR interfaces for VPLs can potentially aid authors by highlighting either related links or potential nodes rather than over-relying on a palette metaphor where the available nodes require a paging system to manage. Current dataflow representations are particularly vulnerable to the sheer quantity of visual wires, which make be further reduced by varying the opacity of the wires or providing additional hints as to which wires are currently of interest based on gaze or hover events. A particularly important direction for research is to increase the utility of refactoring or insertion suggestions.

Debugging features: Another key feature for understandability is visualizing the execution sequence of a graph and the data that is passed along each port. This would enable participants to not only step through the execution, but to record user actions in order to repeat those inputs with a different graph configuration.

4 CONCLUSION

RealityFlow offers one path for researchers interested in comparing immersive authoring approaches and interactions using visual programming languages. But it will require community effort to maintain and extend, and so it must address community needs. An open-source solution would make research in VR authoring more accessible, as it is critical to control experimental conditions and collect usage data from users, which is nearly impossible while adhering to the current terms of service of the commercial products mentioned here. Open access projects offer an opportunity to not just lower barriers and increase access to immersive technologies, but to ensure those technologies achieve their potential as a new form of computing and not just as a new form of consumption or user-generated content strategy. Further efforts to ensure open-source immersive authoring is successful are critical to the advancement of the field, whether through studies on the authoring behavior, interactions or visualizations.

ACKNOWLEDGEMENTS

The author wishes to thank the members of the three CS senior design teams at UCF that worked on the project over the past two years and contributed time and engineering effort toward the goals described herein, including Victor Solal, Owen Mak, John Lynch, Morgan Wilkinson, Faisal Salameh, Philip Bettler, Nyasha Frank, Matthew Kurtz, Yash Bhosale, Alexis Lambert, Jonathan Havener, Jonathan White, Kayden Wilkinson, Manuel Govea, and Stephen Rigdon.

² <https://github.com/alelievr/NodeGraphProcessor>

³ <https://github.com/many-realities-studio/realityflow>

REFERENCES

- [1] A. Salter and J. Murray, *Flash: Building the Interactive Web*. MIT Press, 2014. Accessed: Oct. 06, 2014. [Online]. Available: <http://books.google.com/books?hl=en&lr=&id=hhJmBAAQBAJ&pgis=1>
- [2] M. Kordaki, I. Berdousis, I. Berdousis, and Dept of Cultural Technology and Communication, University of the Aegean, "IDENTIFYING BARRIERS FOR WOMEN PARTICIPATION IN COMPUTER SCIENCE," *PEJES*, vol. 2, no. 2, pp. 5–20, Jan. 2020, doi: 10.26520/pejes.2020.2.2.5-20.
- [3] J. M. Wing, "Computational thinking," *Communications of the ACM*, vol. 49, no. 3, pp. 33–35, 2006.
- [4] A. Gomes and A. J. Mendes, "Learning to program-difficulties and solutions," in *International conference on engineering Education-ICEE*, 2007, vol. 2007.
- [5] B. D. Boulay, "Some Difficulties of Learning to Program:," *Journal of Educational Computing Research*, Jan. 1995, doi: 10.2190/3LFX-9RRF-67T8-UVK9.
- [6] A. Garcia and E. Morrell, "City youth and the pedagogy of participatory media," *Learning, Media and Technology*, vol. 38, no. 2, pp. 123–127, Jun. 2013, doi: 10.1080/17439884.2013.782040.
- [7] F. Pallavicini, A. Pepe, and M. E. Minissi, "Gaming in Virtual Reality: What Changes in Terms of Usability, Emotional Response and Sense of Presence Compared to Non-Immersive Video Games?," *Simulation & Gaming*, vol. 50, no. 2, pp. 136–159, Apr. 2019, doi: 10.1177/1046878119831420.
- [8] J. T. Murray and E. K. Johnson, *XR Content Authoring Challenges: The Creator-Developer Divide*. CRC Press, 2021, pp. 245–264. doi: 10.1201/9781003052838-16.
- [9] G. A. Lee, C. Nelles, M. Billingham, and G. J. Kim, "Immersive Authoring of Tangible Augmented Reality Applications," in *Proceedings of the 3rd IEEE/ACM International Symposium on Mixed and Augmented Reality*, USA, Nov. 2004, pp. 172–181. doi: 10.1109/ISMAR.2004.34.
- [10] L. Zhang and S. Oney, "FlowMatic: An Immersive Authoring Tool for Creating Interactive Scenes in Virtual Reality," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, 2020. Accessed: Dec. 08, 2020. [Online]. Available: <https://dl.acm.org/action/showBook?doi=10.1145/3313831>
- [11] D. Kao et al., "Hack.VR: A Programming Game in Virtual Reality," *arXiv:2007.04495 [cs]*, Nov. 2020, Accessed: Dec. 07, 2020. [Online]. Available: <http://arxiv.org/abs/2007.04495>
- [12] M. Erwig, K. Smeltzer, and X. Wang, "What is a visual language?," *Journal of Visual Languages & Computing*, vol. 38, pp. 9–17, Feb. 2017, doi: 10.1016/j.jvlc.2016.10.005.
- [13] B. A. Myers, "Taxonomies of visual programming and program visualization," *Journal of Visual Languages & Computing*, vol. 1, no. 1, pp. 97–123, Mar. 1990, doi: 10.1016/S1045-926X(05)80036-9.
- [14] G. D. Kuh, "What Student Affairs Professionals Need to Know About Student Engagement," *Journal of College Student Development*, vol. 50, no. 6, pp. 683–706, Nov. 2009, doi: 10.1353/csd.0.0099.
- [15] A. Repenning, "Moving Beyond Syntax: Lessons from 20 Years of Blocks Programing in AgentSheets," *VLSS*, vol. 3, no. 1, pp. 68–91, Jul. 2017, doi: 10.18293/VLSS2017-010.
- [16] J. Maloney, M. Resnick, N. Rusk, B. Silverman, and E. Eastmond, "The Scratch Programming Language and Environment," *ACM Trans. Comput. Educ.*, vol. 10, no. 4, pp. 1–15, Nov. 2010, doi: 10.1145/1868358.1868363.
- [17] N. Fraser, "Ten things we've learned from Blockly," in *2015 IEEE Blocks and Beyond Workshop (Blocks and Beyond)*, Atlanta, GA, USA, Oct. 2015, pp. 49–50. doi: 10.1109/BLOCKS.2015.7369000.
- [18] *NodeRED*. IBM, 2021.
- [19] M. Andujar, L. Jimenez, J. Shah, and P. Morreale, "Evaluating visual programming environments to teach computing to minority high school students," *Journal of Computing Sciences in Colleges*, vol. 29, no. 2, pp. 140–148, Dec. 2013.
- [20] "Bolt visual scripting is now included in all Unity plans." <https://blogs.unity3d.com/2020/07/22/bolt-visual-scripting-is-now-included-in-all-unity-plans/> (accessed Dec. 14, 2020).
- [21] S. C. Pokress and J. J. D. Veiga, "MIT App Inventor: Enabling Personal Mobile Computing," *arXiv:1310.2830 [cs]*, Oct. 2013, Accessed: May 17, 2021. [Online]. Available: <http://arxiv.org/abs/1310.2830>
- [22] J. Kuhn, "Minecraft: Education Edition," *CJ*, vol. 35, no. 2, pp. 214–223, Nov. 2017, doi: 10.1558/cj.34600.
- [23] K. Karbotte, "NeosVR Lets You Build Virtual Worlds With Others, Remotely And In Real Time | Tom's Hardware," Oct. 01, 2016. <https://www.tomshardware.com/news/neosvr-world-building-engine-devlog,32792.html> (accessed Jul. 08, 2020).
- [24] B. Fry and C. Reas, "Processing: programming for the media arts," *AI & Soc*, vol. 20, pp. 526–538, 2006, doi: 10.1007/s00146-006-0050-9.
- [25] B. MacIntyre and T. F. Smith, "Thoughts on the Future of WebXR and the Immersive Web," in *2018 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, Oct. 2018, pp. 338–342. doi: 10.1109/ISMAR-Adjunct.2018.00099.
- [26] B. Shen, W. Tan, J. Guo, H. Cai, B. Wang, and S. Zhuo, "A Study on Design Requirement Development and Satisfaction for Future Virtual World Systems," *Future Internet*, vol. 12, no. 7, p. 112, Jul. 2020, doi: 10.3390/fi12070112.
- [27] B. Lang, "Microsoft Reveals 'Maquette', a VR Tool for Spatial Ideation and Design," *Road to VR*, Jan. 16, 2019. <https://www.roadtovr.com/microsoft-reveals-maquette-a-vr-tool-for-spatial-ideation-and-design/> (accessed Jul. 08, 2020).
- [28] R. Gonzalez, "With Blocks, Google Helps Solve One of VR's Biggest Problems | WIRED," *Wired*, Jul. 06, 2017. <https://www.wired.com/story/google-blocks-vr/> (accessed Jul. 08, 2020).